

2.7 - Serverless: modelo e casos de uso

Código que roda sem servidor para gerenciar

O modelo de execução serverless

FaaS no centro, BaaS ao redor

- **Você escreve funções, a plataforma executa sob demanda**
 - Nada de provisionar, escalar ou manter servidor
- **Escala automática do zero ao pico**
 - A plataforma cria e destrói instâncias por você
- **Pagamento apenas pelo tempo de execução**
 - Função parada não gera custo nenhum
- **FaaS é o núcleo, BaaS completa o ecossistema**
 - Banco, fila e auth gerenciados ao redor da função

Servidor gerenciado vs. função sob demanda

Servidor ou container

- Você provisiona e mantém a capacidade
- Paga mesmo quando está ocioso
- Escala é responsabilidade sua
- Controle total sobre o ambiente

Função serverless

- A plataforma provisiona por você
- Paga só pelo que executa
- Escala automática e gerenciada
- Menos controle, menos operação

Onde o serverless se destaca

- **Workloads orientados a evento com tráfego irregular**
 - Picos esporádicos sem servidor ocioso entre eles
- **Processamento assíncrono de arquivos e tarefas**
 - Gerar relatório, redimensionar imagem, tratar upload
- **Automações e integrações pontuais**
 - Glue code que reage a eventos da própria nuvem
- **APIs de baixo volume e baixa criticidade**
 - Quando a latência de cold start é aceitável

Serverless brilha em carga intermitente

- Carga contínua e previsível raramente compensa em FaaS
- Carga intermitente e sem estado é o caso ideal

Use serverless quando o trabalho é esporádico, sem estado e tolera alguma latência. Para fluxo constante de alto volume, container dedicado costuma sair mais barato.

- A conta muda com o perfil de tráfego, calcule antes

Lambda para relatórios PDF: 70% mais barato

- **Geração de relatório é pesada, irregular e esporádica**
 - Poucas execuções por dia, por transportadora

LUNALOG

A LunaLog usa Lambda para gerar relatórios de entrega em PDF: um processo pesado, irregular, executado poucas vezes por dia por transportadora. Nenhum servidor fica ocioso esperando o próximo relatório. O custo da função serverless ficou 70% menor do que manter uma instância EC2 dedicada para a tarefa. O caso mostra exatamente onde serverless brilha: cargas intermitentes, sem estado, com tempo de processamento tolerável. Foi a escolha óbvia, e os números confirmaram.



Serverless parece a solução perfeita para muitas situações, e em várias delas é mesmo. Mas toda abstração poderosa cobra um preço que não aparece no catálogo. Existe uma latência escondida, um limite de tempo e uma forma sutil de ficar preso a uma plataforma. O que o serverless não te conta antes de você adotar?

Serverless: cold starts, estado e lock-in